Department of Electrical and Computer Engineering
University of Victoria

ECE 355 – Microprocessor-Based Systems

Project: PWM Signal Generation and Monitoring System

Submitted: November 22nd, 2024

Lab: B05

Hoang Tuan Kiet Vu –

Minh Nguyen -

Contents

List of Figures

List of Table

Problem Description/Specifications

The main objective is to develop an embedded system that monitors and controls a Pulse-Width-Modulated (PWM) signal from a 555 Timer (NE555 IC). Below are some criteria that must be adopted:

1. The SMF0 Discovery board must be used to control the PWM Signal frequency and duty cycle.
2. The microcontroller will measure the voltage across the potentiometer (POT) on the PBMCUSLK (provided by the university) board to manipulate the PWM Signal Frequency.
3. The frequency of the 555 Timer signal, and the frequency of the Function Generator signal must be calculated and measured by the microcontroller.
4. A USER Button must be used to toggle which frequency is being measured by the microcontroller.
5. All of the information (both frequencies and POT resistance) must be displayed on the PBMCUSLK board's LED Display (SSD1306 IC).

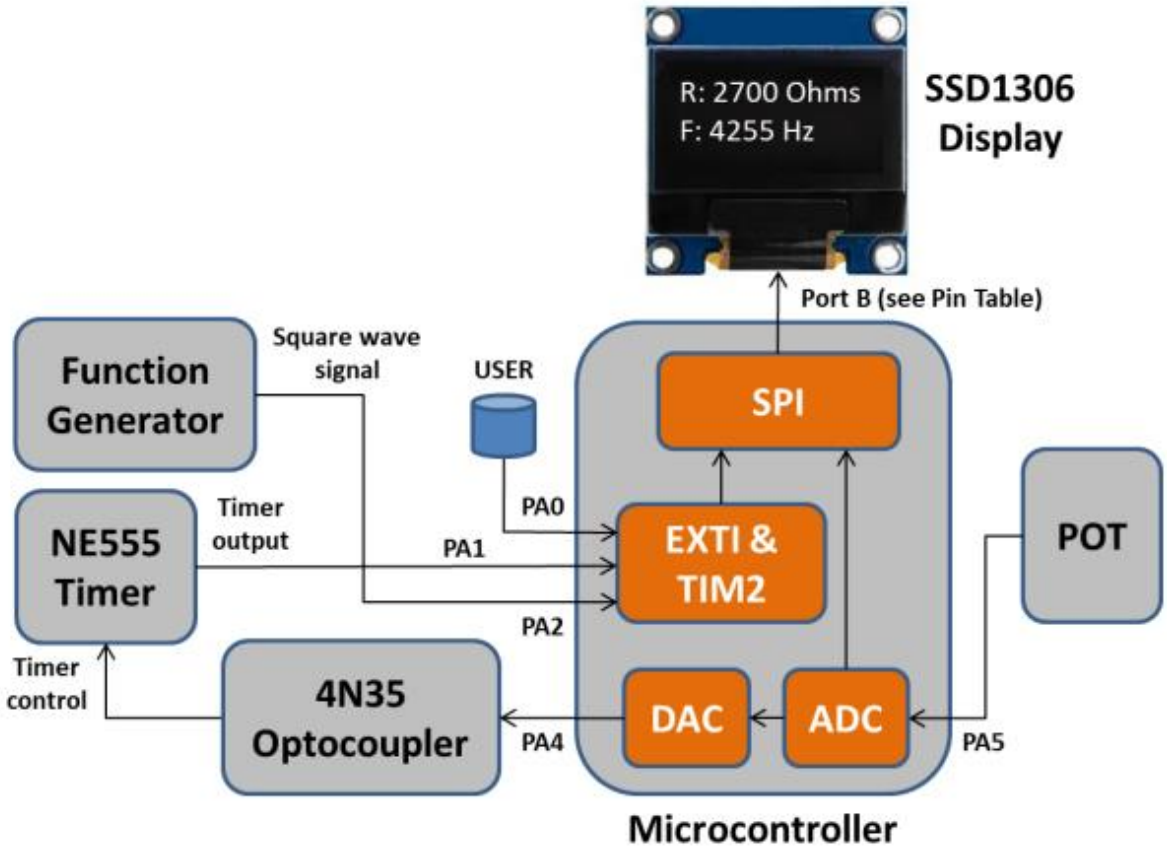

Figure 1: Basic diagram of the project.

For this project, both the Function Generator and the 555 Timer will generate a separate square-wave signal to the board's input. The microcontroller will raise the corresponding interrupt handler to switch between the two states whenever the USER Button is pressed. The primary task would be to output both frequencies to the OLED display. Below are all the specified Inputs and Outputs that we must follow:

Table 1: Buttons and Pins Specifications.

STM32F0	Signal	Direction	Comments
PA0	USER Button	Input	Must be used with EXTI0 for the interrupt
PA1	555 Timer	Input	Must be used with EXTI1 to measure the frequency of the 555 Timer
PA2	Function Generator	Input	Must be used with EXTI2 to measure the frequency of the 555 Timer
PA4	DAC	Output (Analog)	Digital-to-Analog Converter, Optocoupler will be used to control the signal frequency and duty cycle of the 555 Timer
PA5	ADC	Input (Analog	Analog-to-Digital Converter will be used to output the voltage read from the potentiometer on the PBMCUSLK board and feed to the input of the DAC. The ADC's task is to monitor that voltage signal and calculated the corresponding potentiometer resistance value.
PB3	SCLK	Output (AF0)	(Display D0: "Serial Clock" = SPI SCK)
PB4	RES#	Output	(Display "Reset")
PB5	SDIN	Output (AF0)	(Display D1: "Serial Data" = SPI MOSI)
PB6	CS#	Output	(Display "Chip Select")
PB7	D/C#	Output	(Display "Data/Command")
PC8	Blue LED	Output	
PC9	Green LED	Output	
OLED	Display	Output	The OLED will display the frequency and resistance of the current signal, the button will switch between the 555 Timer's signal or the Function Generator's signal

Design/Solution

We designed and implemented software and hardware aspects as the two components of the whole system. The hardware covers the interactions between software and electrical setup, while the software covers all the algorithms based on the setup to generate appropriate outputs when receiving hardware inputs.

Software

Below is the detailed flowchart that captures all the steps the program takes for the tasks. This will only cover the coding-side of the project; the hardware side will be covered later in the project.

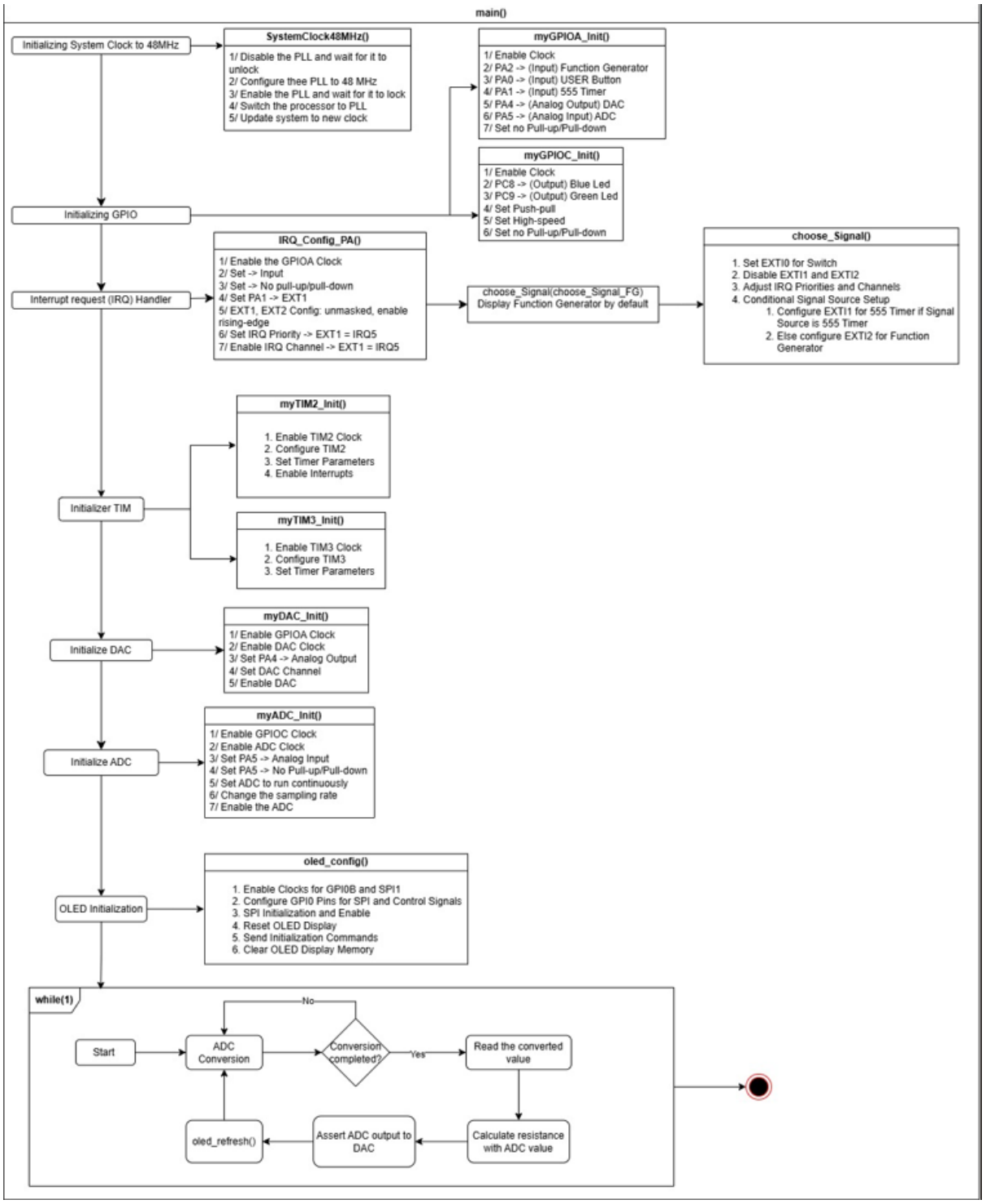

Figure 2: Flowchart illustrating the algorithms of main().

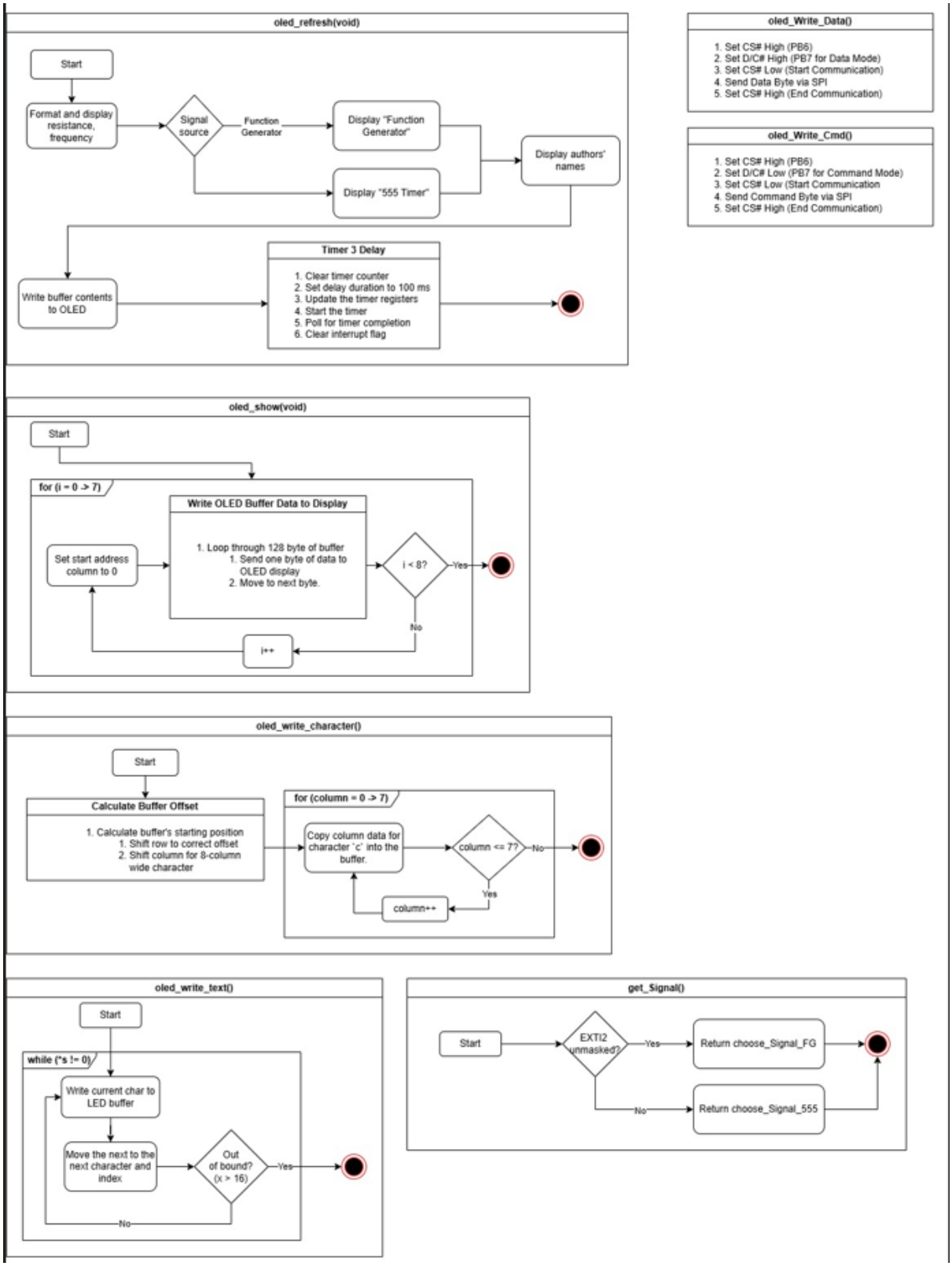

Figure 3: Flowchart illustrating the algorithms of OLEDs and Timers.

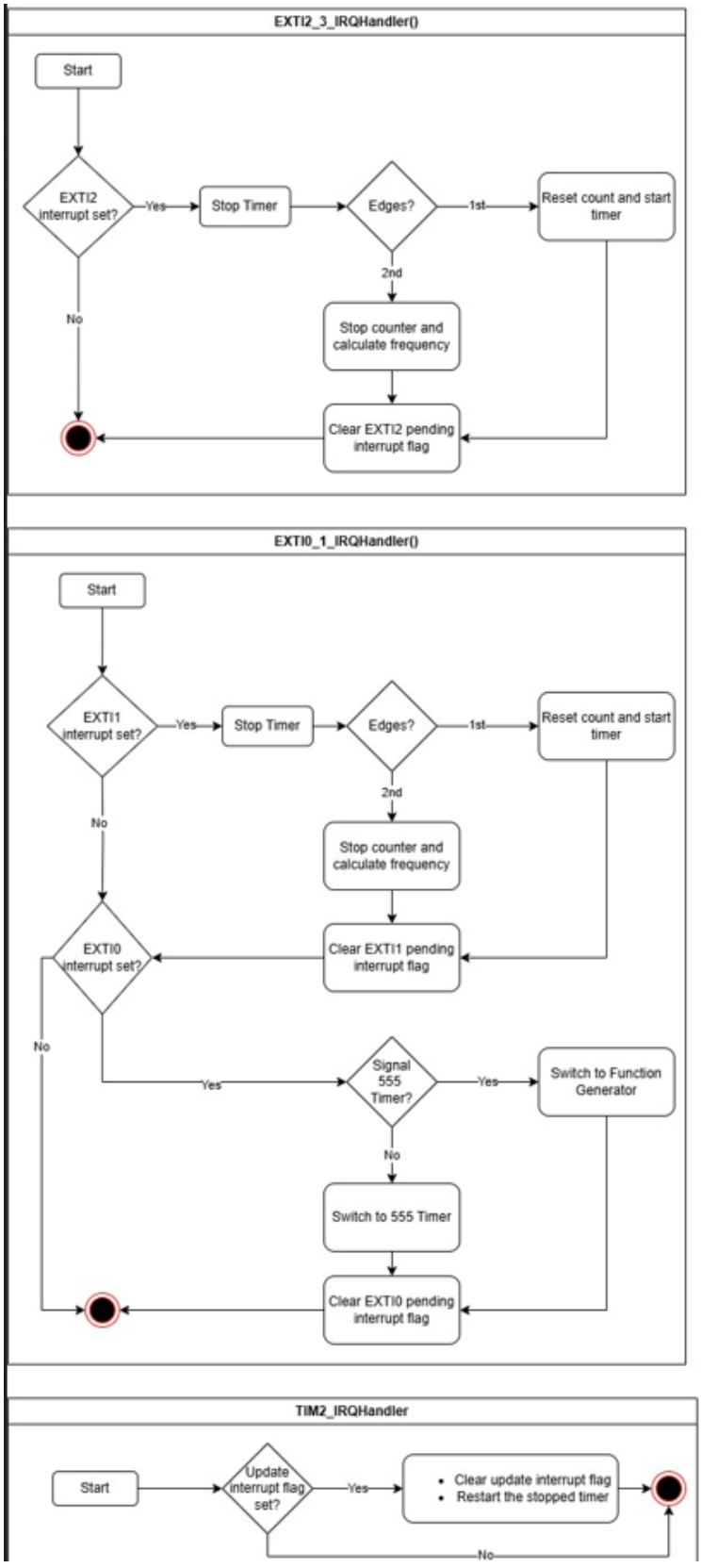

Figure 4: Flowchart illustrating the algorithms of IRQ and EXTI.

To sum up, in basic terms, our system will start by measuring and calculating the frequency of the Function Generator (similar to part 2 of the lab). After receiving the expected measurements, the program will output it to the OLED display accordingly. The system will then loop in this state, constantly calculating the frequency of the signal and outputting it to the display until a Button is pressed, this will trigger interruptions and force the microcontroller to switch to a different signal input. As for the frequency, our logic is two raising interruptions between the two rising-edge, from this plus information about the timer, we can derive the frequency. More detailed explanation is provided in the code, with explanation for each function.

Hardware

For this project, the main hardware component is the STMF0 Discovery Board, used to measures different frequencies based on button inputs. Figures 6 and 5 show the actual implementations used during the project. The STM32F0 has a baseboard that includes buttons for submitting inputs and can be connected to computer to transfer information for the algorithms. The main board also feature a SSD1306 LED Display screen to display output from computer program. Below is a detailed schematic outlining how 4N35 and NE555 components are to be connected in the STM32F0's breadboard the project to work.

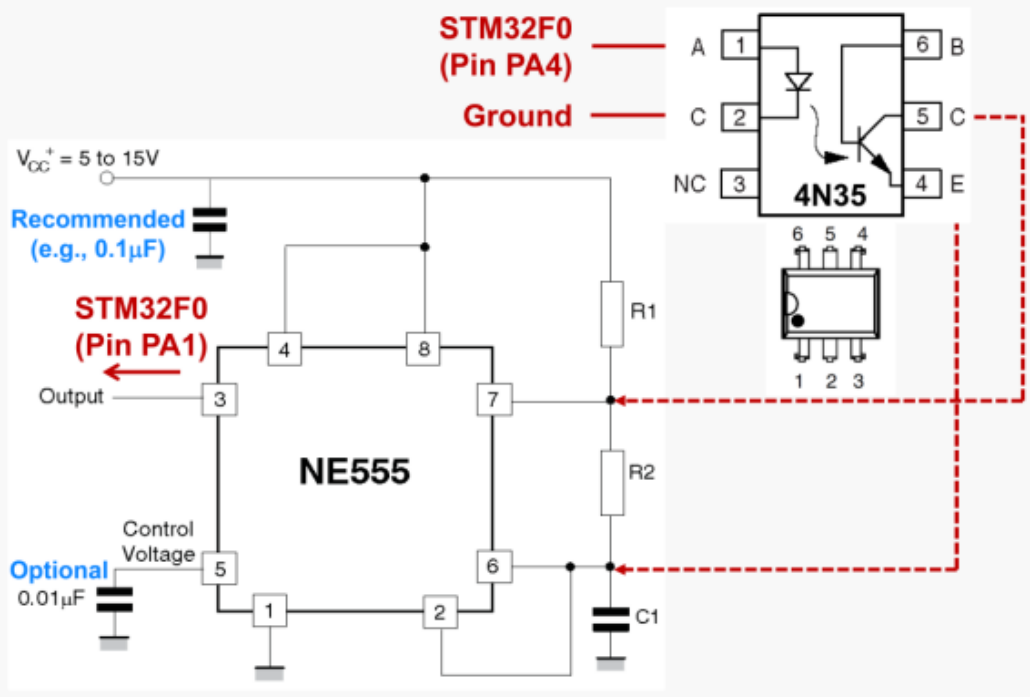

Figure 5: Electrical diagram of the NE555 and 4N353.

The NE555 is an integrated circuit that provides a precision Timer with accurate time delays or oscillation. The 4N35 is an Optocoupler or semiconductor device that filters unwanted signals by separating high and low-voltage systems.

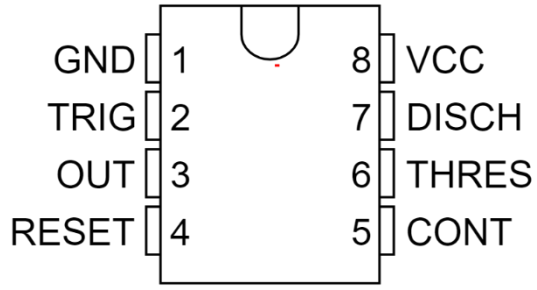

Figure 7: NE555 pins layout.

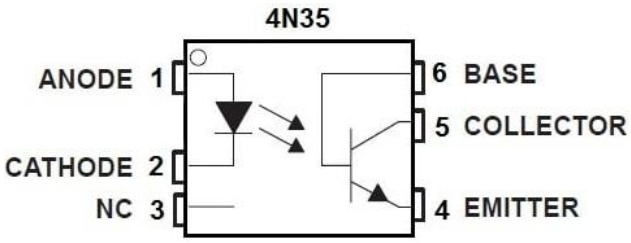

Figure 6: 4N35 pins layout.

Capacitors are also recommended to help the circuit flow better and avoid any unwanted reverse current flow.

Apart from these connections, physical pins the board must also be connected:

- PB3 (SCK) - J5 pin 21
- PB4 (RES#) - J5 pin 19
- PB5 (MOSI) - J5 pin 17
- PB6 (CS#) - J5 pin 25
- PB7 (D/C#) - J5 pin 23

While testing and troubleshooting the projects, we came across several roadblocks with setting both Eclipse and the hardware to work smoothly. However, the lab webpage contains information about tips that solve most issues.

1. Don’t use PA13 and PA14 as input because they are already used by the board.
2. Writing bit “1” will clear a flag in the external interrupt **EXTI->PR**.
3. Include SPI Library in the project.

In the first section of the lab, all the wires and connections were implemented by our team. However, as went on, most of the station was already electrically configured. The circuit below was not implemented by us, nor the station was originally for us as we switched stations frequently due to students from ECE 355 and ECE 255 using the same lab. However, each time we came to the lab, the circuit was always checked by us and any problem was fixed accordingly.

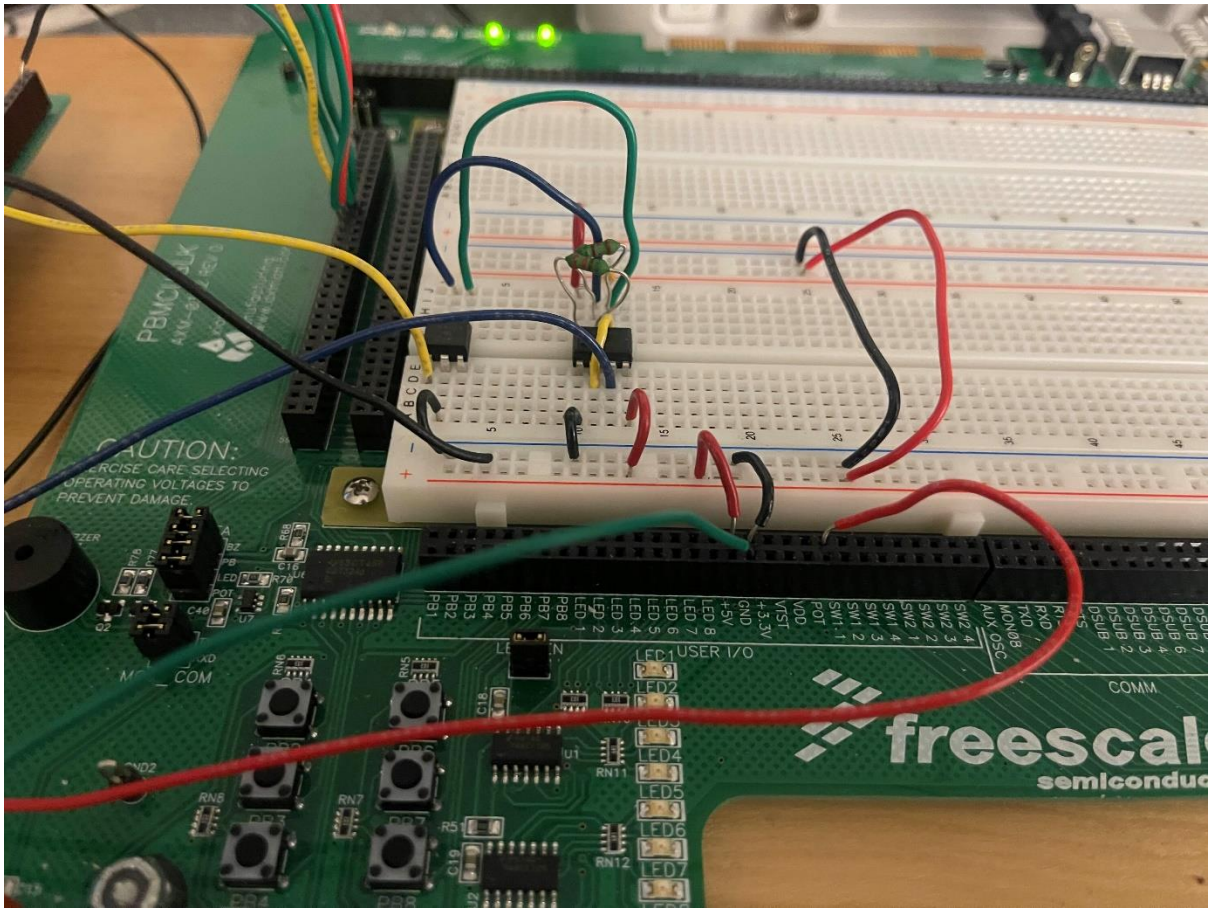

Figure 8: The connected circuit between N555 and 4N35.

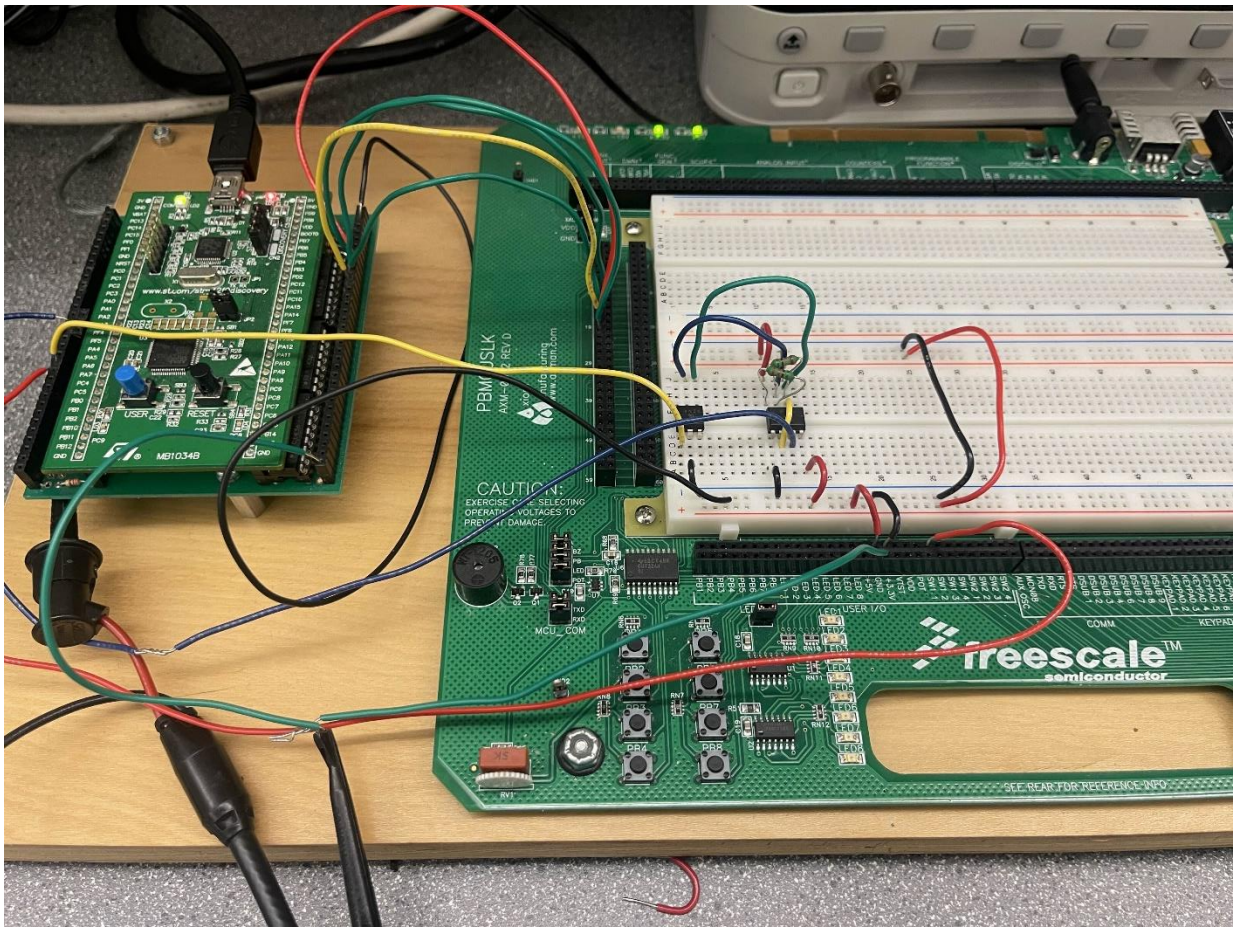

Figure 9: Connections between the Microcontroller and board.

Testing/Results

To test and implement the final project, we followed the procedures that were recommended by our lab's teaching assistant. We first start with Part 2 of the introductory lab as this was the finished function for the Function Generator. For this first part, we also used a multimeter and oscilloscope to make sure we were receiving signals coming to PA2. To verify whether our system works correctly or not, we printed out the output of the frequency on the console and confirmed that it was the same as the Function Generator.

After that, we started implementing the OLED display according to the template "main.c" provided in the lab manual. We start by initializing the correct configurations. We then implement algorithms to try printing anything on the OLED display. After verifying the display indeed works, we move into the next stage.

For the 555 Timer, the algorithm was somewhat similar to the Function Generator part, especially when calculating the frequency according to the rising between two edges. However, the 555 Timers was more complex as it introduced the functionality of the DAC and ADC. To test this part of the project, we also followed the same procedure as testing the Function Generator. After making sure each sub-system worked precisely as planned, we then gathered everything into one large file, this part of the project was the most difficult, as we were having difficulty with alternating between the two inputs. Further information will be detailed in the next part.

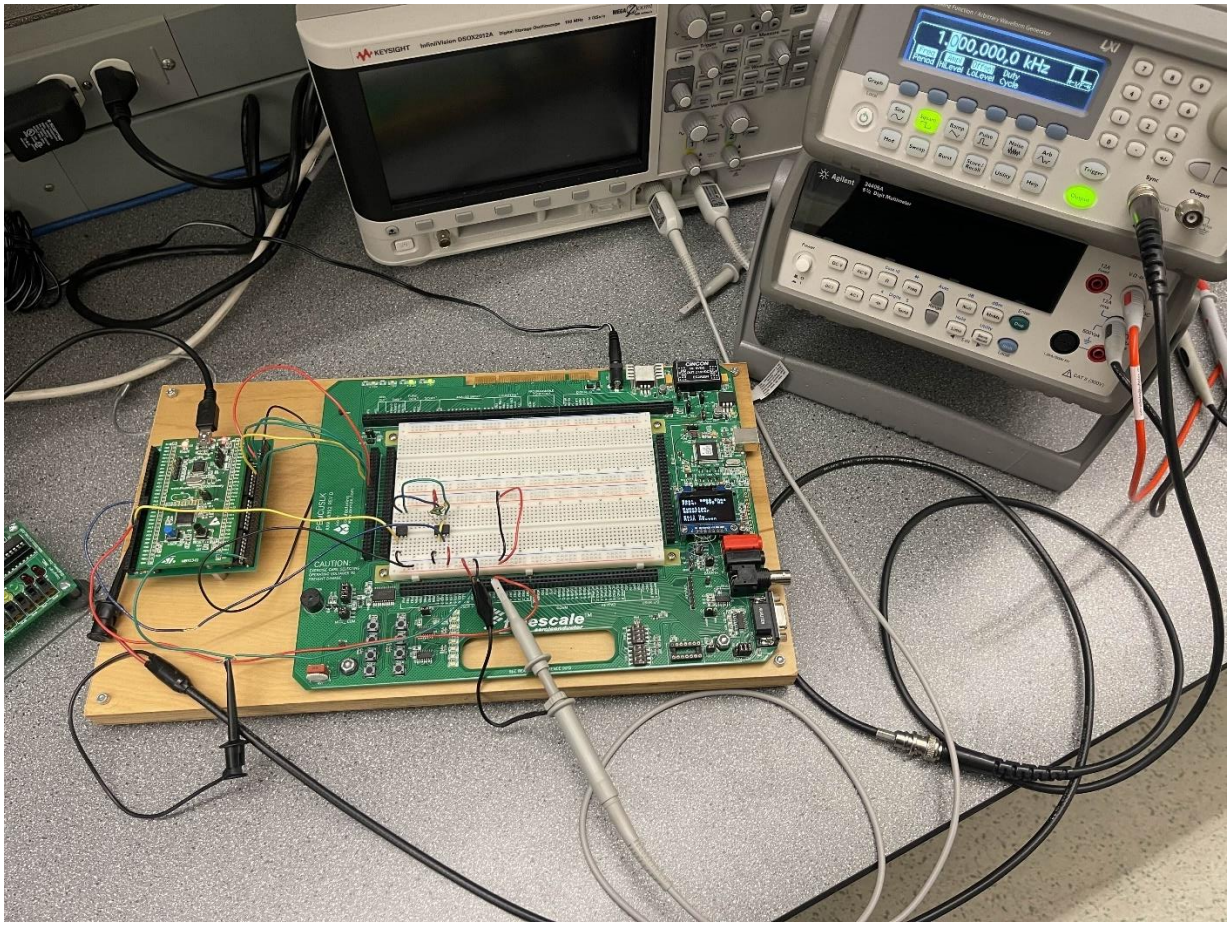

Figure 10: The setup of the test station.

To verify the accuracy of the system, we measured the frequency calculated by the system to the Function Generator.

Table 2: Accuracy between the System and Function Generator.

Function Generator	System	Accuracy
100 Hz	99 Hz	~ 1 %
500 Hz	497 Hz	~ 0.6 %
1 kHz	995 Hz	~ 0.5 %
5 kHz	4987 Hz	~ 0.26 %
10 kHz	10004 Hz	~ 0.04 %
30 kHz	30360 Hz	~ 1.2 %
50 kHz	51227 Hz	~ 2.454 %
80 kHz	83478 Hz	~ 4.3475 %
100 kHz	105494 Hz	~ 5.494 %

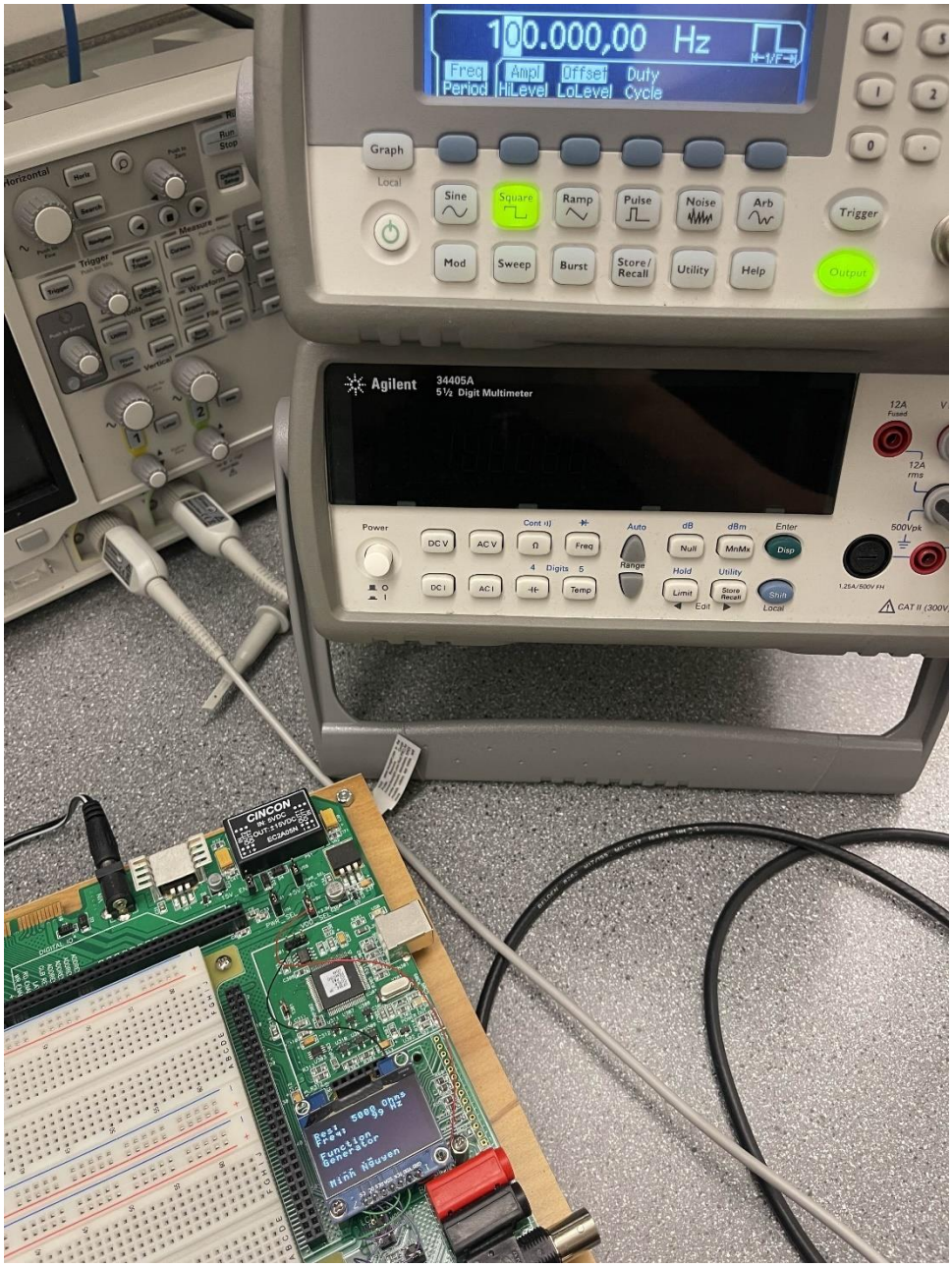

Figure 11: Testing procedure first capture.

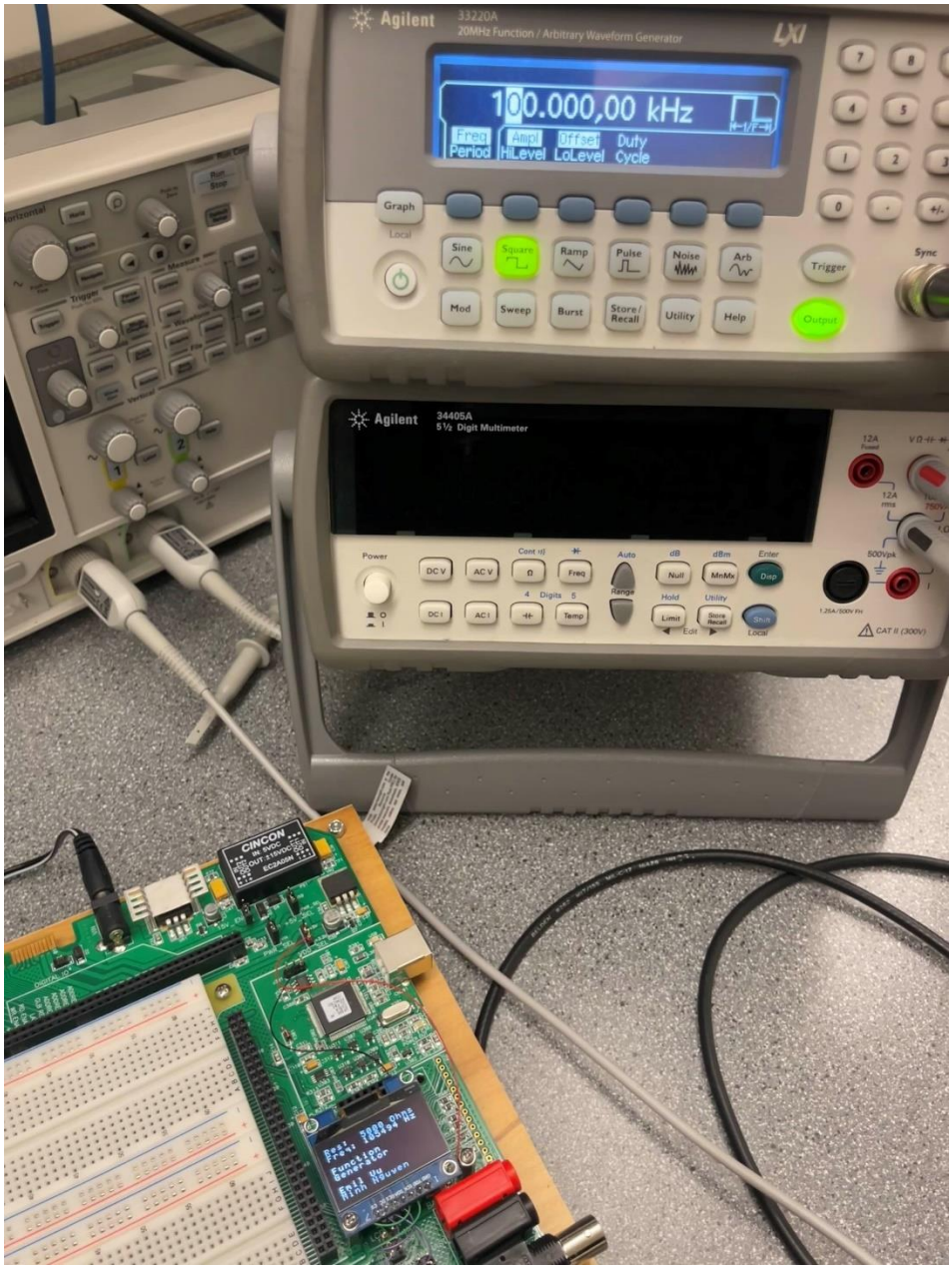

Figure 12: Testing procedure last capture.

Based on our testing, the accuracy of the system begins to dip after 100 kHz. As before this threshold, the accuracy is still within 5%. With this information, we can safely conclude that the algorithms work as intended.

Discussion

Through debugging and troubleshooting errors, we develop major improvements during the project. We first tried putting the interruption flags and User Button detection in the main while loop. We thought that putting the button-pressed detection in the main loops made sense as this is our core functionality. However, this was not the case as it was recommended by one of our teaching assistants that the interruption flag in the microcontroller should be able to do that, because of this, there was no reason to hard code a button detection part in the main while loop. Rather, we should raise an interruption flag each time the button is pressed.

We also experienced multiple difficulties with interruption flags. Our previous version of the code tried to pump in as many flags as possible. We thought that the systems would proceed and rank the importance of the flags and execute it based on the ranking. However, the professor helped us understand that each flag will be put in a list. The microcontroller will then execute the list based on priority. This has helped us solve the issues by adding more functions that govern different tasks and avoid overlapping.

Unfortunately, during the demo lab, we couldn't get the whole system to work together regardless of the sub-systems working. Our program kept getting stuck in the button error as mentioned above. After listening to our teaching assistant's advice and improving the code. We were able to get the whole system to come together as intended for the final project.

Several improvements could be implemented to make the program better. Although we have cleared up a lot of unnecessary functions, we believe that parts of the code could be rewritten to make it easier for others to follow. Although our frequency seems to be correct, as the resolution gets smaller (by increasing a small interval instead of a large one), it seems like the outputs were different from the Function Generator's inputs. We believe better mathematical algorithms could be adopted to increase accuracy.

While finishing up the project, we found these small errors. After disconnecting the Function Generator or deselecting the output. The system will correctly output 0 Hz frequency in the OLED. After we switch to the 555 Timer Signal, the frequency will change accordingly. However, if we hit the switch again, the OLED display will go back to the Function Generator Signal, but instead of displaying 0 Hz, the display outputs the frequency for the 555 Timer Signal.

This could be because we did not reset the screen OLED System after each update. Rather, we overlap the current values displayed on the screen with new values that we want to display. This will work fine if there are always values to be displayed continuously. However, if there is a period such as the mentioned one or when the length of the values is different, incorrect information might be displayed and confuse the users. To tackle this problem, a separate function "oled_reset" or “clear_screen” must be implemented to reset the OLED display to be empty before displaying any new values. This will ensure old values on the display get erased and avoid overlapping

Figure 13: After disconnecting the Function Generator.

Figure 14: After pushing the button to change to the 555 Timer.

Figure 15: After pushing the button to change back to the Function Generator.

Figure 16: After hitting the physical reset button from the board.

Even though hitting the physical reset button
on the microcontroller will force the display to reload and input the correct information, we still believe this is a drawback and could be improved in future revisions.

Appendices

Below is all of the code captured from my VSCode.

```
// Emil Vu            - Minh Nguyen
// This file is part of the GNU ARM Eclipse distribution.
// Copyright (c) 2014 Liviu Ionescu.
//

// ----------------------------------------------------------------------

#include <stdio.h>
#include "diag/Trace.h"
```

```
#include <string.h>

#include "cmsis/cmsis_device.h"

// ----------------------------------------------------------------------
//
// STM32F0 led blink sample (trace via $(trace)).
//
// In debug configurations, demonstrate how to print a greeting message
// on the trace device. In release configurations the message is
// simply discarded.
//
// To demonstrate POSIX retargetting, reroute the STDOUT and STDERR to the
// trace device and display messages on both of them.
//
// Then demonstrates how to blink a led with 1Hz, using a
// continuous loop and SysTick delays.
//
// On DEBUG, the uptime in seconds is also displayed on the trace device.
//
// Trace support is enabled by adding the TRACE macro definition.
// By default the trace messages are forwarded to the $(trace) output,
// but can be rerouted to any device or completely suppressed, by
// changing the definitions required in system/src/diag/trace_impl.c
// (currently OS_USE_TRACE_ITM, OS_USE_TRACE_SEMIHOSTING_DEBUG/_STDOUT).
//
// The external clock frequency is specified as a preprocessor definition
// passed to the compiler via a command line option (see the 'C/C++ General' ->
// 'Paths and Symbols' -> the 'Symbols' tab, if you want to change it).
// The value selected during project creation was HSE_VALUE=48000000.
//
/// Note: The default clock settings take the user defined HSE_VALUE and try
// to reach the maximum possible system clock. For the default 8MHz input
// the result is guaranteed, but for other values it might not be possible,
// so please adjust the PLL settings in system/src/cmsis/system_stm32f0xx.c
//


// ----- main() ---------------------------------------------------------

// Sample pragmas to cope with warnings. Please note the related line at
// the end of this function, used to pop the compiler diagnostics status.
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-parameter"
#pragma GCC diagnostic ignored "-Wmissing-declarations"
#pragma GCC diagnostic ignored "-Wreturn-type"


/*** This is partial code for accessing LED Display via SPI interface. ***/
```

```
// Clock prescaler
#define myTIM2_PRESCALER ((uint16_t)0x0000)
#define myTIM3_PRESCALER (47999)

// Maximum possible setting for overflow
#define myTIM2_PERIOD ((uint32_t)0xFFFFFFFF)
#define myTIM3_PERIOD ((uint32_t)0x0000FFFF)

// #define BAR_GRAPH_OFFSET  ( 512 + 32 )

#define choose_Signal_555 (0)
#define choose_Signal_FG (1)


// Global variables used
unsigned int Freq = 0;  // Example: measured frequency value (global variable)
unsigned int Res = 0;   // Example: measured resistance value (global variable)
uint16_t ADCValue = 0;

unsigned char oled_buffer[1024];
volatile uint32_t ElapsedTime = 0; // Keep track of elapsed time
volatile uint32_t timerTriggered = 0; // Timer triggered for the edge detection

void oled_Write(unsigned char);
void oled_write_text(unsigned int x, unsigned int y, char *s);
void oled_write_character(unsigned int x, unsigned int y, unsigned int c);

void oled_config(void);
void oled_show();
void oled_refresh(void);
void choose_Signal(uint16_t Source);
uint16_t get_Signal(void);

static void myDAC_Init(void);
static void myADC_Init(void);

void IRQ_Config_PA(void);
void myGPIOA_Init(void);
void myGPIOC_Init(void);
void myTIM2_Init(void);
void myTIM3_Init(void);

SPI_HandleTypeDef SPI_Handle;


//
// LED Display initialization commands
//
```

```
unsigned char oled_init_cmds[] =
{
    0xAE,
    0x20, 0x00,
    0x40,
    0xA0 | 0x01,
    0xA8, 0x40 - 1,
    0xC0 | 0x08,
    0xD3, 0x00,
    0xDA, 0x32,
    0xD5, 0x80,
    0xD9, 0x22,
    0xDB, 0x30,
    0x81, 0xFF,
    0xA4,
    0xA6,
    0xAD, 0x30,
    0x8D, 0x10,
    0xAE | 0x01,
    0xC0,
    0xA0
};


//
// Character specifications for LED Display (1 row = 8 bytes = 1 ASCII character)
// Example: to display '4', retrieve 8 data bytes stored in Characters[52][X] row
//          (where X = 0, 1, ..., 7) and send them one by one to LED Display.
// Row number = character ASCII code (e.g., ASCII code of '4' is 0x34 = 52)
//
unsigned char Characters[][8] = {
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
    {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
```

```
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
   {0b00000000, 0b00000000, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // SPACE
```

```
  {0b00000000, 0b00000000, 0b01011111, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // !
  {0b00000000, 0b00000111, 0b00000000, 0b00000111, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // "
  {0b00010100, 0b01111111, 0b00010100, 0b01111111, 0b00010100,0b00000000, 0b00000000,
0b00000000},  // #
  {0b00100100, 0b00101010, 0b01111111, 0b00101010, 0b00010010,0b00000000, 0b00000000,
0b00000000},  // $
  {0b00100011, 0b00010011, 0b00001000, 0b01100100, 0b01100010,0b00000000, 0b00000000,
0b00000000},  // %
  {0b00110110, 0b01001001, 0b01010101, 0b00100010, 0b01010000,0b00000000, 0b00000000,
0b00000000},  // &
  {0b00000000, 0b00000101, 0b00000011, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // '
  {0b00000000, 0b00011100, 0b00100010, 0b01000001, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // (
  {0b00000000, 0b01000001, 0b00100010, 0b00011100, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // )
  {0b00010100, 0b00001000, 0b00111110, 0b00001000, 0b00010100,0b00000000, 0b00000000,
0b00000000},  // *
  {0b00001000, 0b00001000, 0b00111110, 0b00001000, 0b00001000,0b00000000, 0b00000000,
0b00000000},  // +
  {0b00000000, 0b01010000, 0b00110000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // ,
  {0b00001000, 0b00001000, 0b00001000, 0b00001000, 0b00001000,0b00000000, 0b00000000,
0b00000000},  // -
  {0b00000000, 0b01100000, 0b01100000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // .
  {0b00100000, 0b00010000, 0b00001000, 0b00000100, 0b00000010,0b00000000, 0b00000000,
0b00000000},  // /
  {0b00111110, 0b01010001, 0b01001001, 0b01000101, 0b00111110,0b00000000, 0b00000000,
0b00000000},  // 0
  {0b00000000, 0b01000010, 0b01111111, 0b01000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // 1
  {0b01000010, 0b01100001, 0b01010001, 0b01001001, 0b01000110,0b00000000, 0b00000000,
0b00000000},  // 2
  {0b00100001, 0b01000001, 0b01000101, 0b01001011, 0b00110001,0b00000000, 0b00000000,
0b00000000},  // 3
  {0b00011000, 0b00010100, 0b00010010, 0b01111111, 0b00010000,0b00000000, 0b00000000,
0b00000000},  // 4
  {0b00100111, 0b01000101, 0b01000101, 0b01000101, 0b00111001,0b00000000, 0b00000000,
0b00000000},  // 5
  {0b00111100, 0b01001010, 0b01001001, 0b01001001, 0b00110000,0b00000000, 0b00000000,
0b00000000},  // 6
  {0b00000011, 0b00000001, 0b01110001, 0b00001001, 0b00000111,0b00000000, 0b00000000,
0b00000000},  // 7
  {0b00110110, 0b01001001, 0b01001001, 0b01001001, 0b00110110,0b00000000, 0b00000000,
0b00000000},  // 8
```

```
  {0b00000110, 0b01001001, 0b01001001, 0b00101001, 0b00011110,0b00000000, 0b00000000,
0b00000000},  // 9
  {0b00000000, 0b00110110, 0b00110110, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // :
  {0b00000000, 0b01010110, 0b00110110, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // ;
  {0b00001000, 0b00010100, 0b00100010, 0b01000001, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // <
  {0b00010100, 0b00010100, 0b00010100, 0b00010100, 0b00010100,0b00000000, 0b00000000,
0b00000000},  // =
  {0b00000000, 0b01000001, 0b00100010, 0b00010100, 0b00001000,0b00000000, 0b00000000,
0b00000000},  // >
  {0b00000010, 0b00000001, 0b01010001, 0b00001001, 0b00000110,0b00000000, 0b00000000,
0b00000000},  // ?
  {0b00110010, 0b01001001, 0b01111001, 0b01000001, 0b00111110,0b00000000, 0b00000000,
0b00000000},  // @
  {0b01111110, 0b00010001, 0b00010001, 0b00010001, 0b01111110,0b00000000, 0b00000000,
0b00000000},  // A
  {0b01111111, 0b01001001, 0b01001001, 0b01001001, 0b00110110,0b00000000, 0b00000000,
0b00000000},  // B
  {0b00111110, 0b01000001, 0b01000001, 0b01000001, 0b00100010,0b00000000, 0b00000000,
0b00000000},  // C
  {0b01111111, 0b01000001, 0b01000001, 0b00100010, 0b00011100,0b00000000, 0b00000000,
0b00000000},  // D
  {0b01111111, 0b01001001, 0b01001001, 0b01001001, 0b01000001,0b00000000, 0b00000000,
0b00000000},  // E
  {0b01111111, 0b00001001, 0b00001001, 0b00001001, 0b00000001,0b00000000, 0b00000000,
0b00000000},  // F
  {0b00111110, 0b01000001, 0b01001001, 0b01001001, 0b01111010,0b00000000, 0b00000000,
0b00000000},  // G
  {0b01111111, 0b00001000, 0b00001000, 0b00001000, 0b01111111,0b00000000, 0b00000000,
0b00000000},  // H
  {0b01000000, 0b01000001, 0b01111111, 0b01000001, 0b01000000,0b00000000, 0b00000000,
0b00000000},  // I
  {0b00100000, 0b01000000, 0b01000001, 0b00111111, 0b00000001,0b00000000, 0b00000000,
0b00000000},  // J
  {0b01111111, 0b00001000, 0b00010100, 0b00100010, 0b01000001,0b00000000, 0b00000000,
0b00000000},  // K
  {0b01111111, 0b01000000, 0b01000000, 0b01000000, 0b01000000,0b00000000, 0b00000000,
0b00000000},  // L
  {0b01111111, 0b00000010, 0b00001100, 0b00000010, 0b01111111,0b00000000, 0b00000000,
0b00000000},  // M
  {0b01111111, 0b00000100, 0b00001000, 0b00010000, 0b01111111,0b00000000, 0b00000000,
0b00000000},  // N
  {0b00111110, 0b01000001, 0b01000001, 0b01000001, 0b00111110,0b00000000, 0b00000000,
0b00000000},  // O
  {0b01111111, 0b00001001, 0b00001001, 0b00001001, 0b00000110,0b00000000, 0b00000000,
0b00000000},  // P
```

```
  {0b00111110, 0b01000001, 0b01010001, 0b00100001, 0b01011110,0b00000000, 0b00000000,
0b00000000},  // Q
  {0b01111111, 0b00001001, 0b00011001, 0b00101001, 0b01000110,0b00000000, 0b00000000,
0b00000000},  // R
  {0b01000110, 0b01001001, 0b01001001, 0b01001001, 0b00110001,0b00000000, 0b00000000,
0b00000000},  // S
  {0b00000001, 0b00000001, 0b01111111, 0b00000001, 0b00000001,0b00000000, 0b00000000,
0b00000000},  // T
  {0b00111111, 0b01000000, 0b01000000, 0b01000000, 0b00111111,0b00000000, 0b00000000,
0b00000000},  // U
  {0b00011111, 0b00100000, 0b01000000, 0b00100000, 0b00011111,0b00000000, 0b00000000,
0b00000000},  // V
  {0b00111111, 0b01000000, 0b00111000, 0b01000000, 0b00111111,0b00000000, 0b00000000,
0b00000000},  // W
  {0b01100011, 0b00010100, 0b00001000, 0b00010100, 0b01100011,0b00000000, 0b00000000,
0b00000000},  // X
  {0b00000111, 0b00001000, 0b01110000, 0b00001000, 0b00000111,0b00000000, 0b00000000,
0b00000000},  // Y
  {0b01100001, 0b01010001, 0b01001001, 0b01000101, 0b01000011,0b00000000, 0b00000000,
0b00000000},  // Z
  {0b01111111, 0b01000001, 0b00000000, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // [
  {0b00010101, 0b00010110, 0b01111100, 0b00010110, 0b00010101,0b00000000, 0b00000000,
0b00000000},  // back slash
  {0b00000000, 0b00000000, 0b00000000, 0b01000001, 0b01111111,0b00000000, 0b00000000,
0b00000000},  // ]
  {0b00000100, 0b00000010, 0b00000001, 0b00000010, 0b00000100,0b00000000, 0b00000000,
0b00000000},  // ^
  {0b01000000, 0b01000000, 0b01000000, 0b01000000, 0b01000000,0b00000000, 0b00000000,
0b00000000},  // _
  {0b00000000, 0b00000001, 0b00000010, 0b00000100, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // `
  {0b00100000, 0b01010100, 0b01010100, 0b01010100, 0b01111000,0b00000000, 0b00000000,
0b00000000},  // a
  {0b01111111, 0b01001000, 0b01000100, 0b01000100, 0b00111000,0b00000000, 0b00000000,
0b00000000},  // b
  {0b00111000, 0b01000100, 0b01000100, 0b01000100, 0b00100000,0b00000000, 0b00000000,
0b00000000},  // c
  {0b00111000, 0b01000100, 0b01000100, 0b01001000, 0b01111111,0b00000000, 0b00000000,
0b00000000},  // d
  {0b00111000, 0b01010100, 0b01010100, 0b01010100, 0b00011000,0b00000000, 0b00000000,
0b00000000},  // e
  {0b00001000, 0b01111110, 0b00001001, 0b00000001, 0b00000010,0b00000000, 0b00000000,
0b00000000},  // f
  {0b00001100, 0b01010010, 0b01010010, 0b01010010, 0b00111110,0b00000000, 0b00000000,
0b00000000},  // g
  {0b01111111, 0b00001000, 0b00000100, 0b00000100, 0b01111000,0b00000000, 0b00000000,
0b00000000},  // h
```

```
  {0b00000000, 0b01000100, 0b01111101, 0b01000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // i
  {0b00100000, 0b01000000, 0b01000100, 0b00111101, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // j
  {0b01111111, 0b00010000, 0b00101000, 0b01000100, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // k
  {0b00000000, 0b01000001, 0b01111111, 0b01000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // l
  {0b01111100, 0b00000100, 0b00011000, 0b00000100, 0b01111000,0b00000000, 0b00000000,
0b00000000},  // m
  {0b01111100, 0b00001000, 0b00000100, 0b00000100, 0b01111000,0b00000000, 0b00000000,
0b00000000},  // n
  {0b00111000, 0b01000100, 0b01000100, 0b01000100, 0b00111000,0b00000000, 0b00000000,
0b00000000},  // o
  {0b01111100, 0b00010100, 0b00010100, 0b00010100, 0b00001000,0b00000000, 0b00000000,
0b00000000},  // p
  {0b00001000, 0b00010100, 0b00010100, 0b00011000, 0b01111100,0b00000000, 0b00000000,
0b00000000},  // q
  {0b01111100, 0b00001000, 0b00000100, 0b00000100, 0b00001000,0b00000000, 0b00000000,
0b00000000},  // r
  {0b01001000, 0b01010100, 0b01010100, 0b01010100, 0b00100000,0b00000000, 0b00000000,
0b00000000},  // s
  {0b00000100, 0b00111111, 0b01000100, 0b01000000, 0b00100000,0b00000000, 0b00000000,
0b00000000},  // t
  {0b00111100, 0b01000000, 0b01000000, 0b00100000, 0b01111100,0b00000000, 0b00000000,
0b00000000},  // u
  {0b00011100, 0b00100000, 0b01000000, 0b00100000, 0b00011100,0b00000000, 0b00000000,
0b00000000},  // v
  {0b00111100, 0b01000000, 0b00111000, 0b01000000, 0b00111100,0b00000000, 0b00000000,
0b00000000},  // w
  {0b01000100, 0b00101000, 0b00010000, 0b00101000, 0b01000100,0b00000000, 0b00000000,
0b00000000},  // x
  {0b00001100, 0b01010000, 0b01010000, 0b01010000, 0b00111100,0b00000000, 0b00000000,
0b00000000},  // y
  {0b01000100, 0b01100100, 0b01010100, 0b01001100, 0b01000100,0b00000000, 0b00000000,
0b00000000},  // z
  {0b00000000, 0b00001000, 0b00110110, 0b01000001, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // {
  {0b00000000, 0b00000000, 0b01111111, 0b00000000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // |
  {0b00000000, 0b01000001, 0b00110110, 0b00001000, 0b00000000,0b00000000, 0b00000000,
0b00000000},  // }
  {0b00001000, 0b00001000, 0b00101010, 0b00011100, 0b00001000,0b00000000, 0b00000000,
0b00000000},  // ~
  {0b00001000, 0b00011100, 0b00101010, 0b00001000, 0b00001000,0b00000000, 0b00000000,
0b00000000}   // <-
};
```

```
void SystemClock48MHz( void )
{
//
// Disable the PLL
//
  RCC->CR &= ~(RCC_CR_PLLON);
//
// Wait for the PLL to unlock
//
  while (( RCC->CR & RCC_CR_PLLRDY ) != 0 );
//
// Configure the PLL for a 48MHz system clock
//
  RCC->CFGR = 0x00280000;

//
// Enable the PLL
//
  RCC->CR |= RCC_CR_PLLON;

//
// Wait for the PLL to lock
//
  while (( RCC->CR & RCC_CR_PLLRDY ) != RCC_CR_PLLRDY );

//
// Switch the processor to the PLL clock source
//
  RCC->CFGR = ( RCC->CFGR & (~RCC_CFGR_SW_Msk)) | RCC_CFGR_SW_PLL;

//
// Update the system with the new clock frequency
//
  SystemCoreClockUpdate();

}



int
main(int argc, char* argv[])
{

  SystemClock48MHz();

  // Initializing all of the variables and ports
```

```
    // GPIO Initialization
    myGPIOA_Init();
    myGPIOC_Init();

    // IRQ Initialization
    IRQ_Config_PA();

    // TIM Initialization
    myTIM2_Init();
    myTIM3_Init();

    // DAC and ADC Initialization
    myDAC_Init();
    myADC_Init();

    // OLED Initialization
    oled_config();

    while (1)
    {
        ADC1->CR |= ((uint32_t)0x00000004); // Start ADC conversion
        while(!(ADC1->ISR & ((uint32_t)0x00000004))); // Wait until it finishes
        ADCValue = (uint16_t) ADC1->DR; // Read the value converted
        Res = (ADCValue*5000)/0xFFF; // Calculate resistance (in milliohms) from ADC value
        DAC-> DHR12R1  = ADCValue; // Assert the output of ADC to DAC

        oled_refresh(); // refresh the display and print the current value to OLED
    }
}

///  Initialize config  ///
// GPIO config
void myGPIOA_Init()
{
    /* Enable clock for GPIOA peripheral */
    // Relevant register: RCC->AHBENR
    // RCC->AHBENR |= 1<<17; // Bit 17 = IOPAEN
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;

    /* Configure PA2 as input for the FUNCTION GENERATOR*/

    // Relevant register: GPIOA->MODER
    // GPIOA->MODER &= ~((0<<4) | (0<<5)); // pin PA2(bits 4:5) as input (00)
    GPIOA->MODER &= ~(GPIO_MODER_MODER2);
    /* Ensure no pull-up/pull-down for PA2 */
    // Relevant register: GPIOA->PUPDR
    // GPIOA->PUPDR &= ~((0<<4) | (0<<5)); // pin PA2 bits (4:5) set no pull down
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR2);
```

```
    /* Configure PA0 as input for the USER BUTTON */
    GPIOA->MODER &= ~(GPIO_MODER_MODER0);
    /* Ensure no pull-up/pull-down for PA0 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR0);

    /* Configure PA1 as input for the 555 Timer */
    GPIOA->MODER &= ~(GPIO_MODER_MODER1);
    /* Ensure no pull-up/pull-down for PA1 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR1);

    /* Configure PA4 as ANALOG OUTPUT for the DAC*/
    GPIOA->MODER |= (GPIO_MODER_MODER4);
    /* Ensure no pull-up/pull-down for PA4 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR4);

    /* Configure PA5 as ANALOG INPUT for the ADC*/
    GPIOA->MODER |= (GPIO_MODER_MODER5_Msk); // Check again
    /* Ensure no pull-up/pull-down for PA5 */
    GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPDR5);
}

void IRQ_Config_PA(void)
{
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN; // Enable GPIOA clock
    GPIOA->MODER &= 0xFFFFFFF3; // input mode
    GPIOA->PUPDR &= 0xFFFFFFF3; // no pull-up/pull-down

    GPIOA->MODER &= 0xFFFFFFCF; // input mode
    GPIOA->PUPDR &= 0xFFFFFFCF; // no pull-up/pull-down

    SYSCFG->EXTICR[0] = 0x00000000; // Set PA1 to the EXTI1 line

    // Configure EXTI2 line
    EXTI->IMR |= ((uint32_t)0x00000002); // unmasked
    EXTI->RTSR |= ((uint32_t)0x00000002); // Enable rising-edge

    // Configure EXTI1 line
    EXTI->IMR |= ((uint32_t)0x00000001); // unmasked
    EXTI->RTSR |= ((uint32_t)0x00000001); // Enable rising-edge

    NVIC->IP[1] = ((uint32_t)0xFFFF00FF); // Set IRQ Priority for EXTI1 = IRQ5
    NVIC->ISER[0] = (uint32_t)0X00000020; // Enable IRQ Channel for EXTI1 = IRQ5

    choose_Signal(choose_Signal_FG); // By default display Function Generator first
}

void myGPIOC_Init()
```

```
{
    /* Enable clock for GPIOC peripheral */
    RCC->AHBENR |= RCC_AHBENR_GPIOCEN;

    /* Configure PC8 as outputs */
    GPIOC->MODER |= GPIO_MODER_MODER8_0;
    /* Ensure push-pull for PC8 */
    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT_8);
    /* Ensure high-speed for PC8 */
    GPIOC->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR8);
    /* Ensure no pull-up/pull-down for PC8 */
    GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPDR8);

    /* Configure PC9 as outputs */
    GPIOC->MODER |= GPIO_MODER_MODER9_0;
    /* Ensure push-pull for PC9 */
    GPIOC->OTYPER &= ~(GPIO_OTYPER_OT_9);
    /* Ensure high-speed mode for PC9 */
    GPIOC->OSPEEDR |= (GPIO_OSPEEDER_OSPEEDR9);
    /* Ensure no pull-up/pull-down for PC9 */
    GPIOC->PUPDR &= ~(GPIO_PUPDR_PUPDR9);
}

void choose_Signal(uint16_t Source)
{
    // Set the EXTI0 line for the switch
    EXTI->IMR |= ((uint32_t)0x00000001); // unmasked
    EXTI->RTSR |= ((uint32_t)0x00000001); // Rising-edge trigger enabled

    // Set the EXTI1 line for the 555 timer
    EXTI->IMR &= ~((uint32_t)0x00000006);   // Disable both inputs
    EXTI->RTSR &= ~((uint32_t)0x00000006);

    // Set priority of IRQ6 to max
    NVIC->IP[1] |= ((uint32_t)0x00FF0000 );

    // Disable IRQ Channel for EXTI1 = IRQ6
    NVIC->ISER[0] &= ~(uint32_t)0X00000040;

    // Set IRQ Priority for EXTI1 = IRQ5
    NVIC->IP[1] = ((uint32_t)0xFFFF00FF);

    // Enable IRQ Channel for EXTI1 = IRQ5
    NVIC->ISER[0] = (uint32_t)0X00000020;

    if (choose_Signal_555 == Source){
        // Set EXTI1 line for the 555 timer
        EXTI->IMR |= ((uint32_t)0x00000002); // unmasked
```

```
        EXTI->RTSR |= ((uint32_t)0x00000002); // Rising-edge trigger enabled

        // Set IRQ Priority for EXTI1 = IRQ5
        NVIC->IP[1] = ((uint32_t)0xFFFF00FF);

        // Enable IRQ Channel for EXTI1 = IRQ5
        NVIC->ISER[0] = (uint32_t)0X00000020;
    } else {

        // Set EXTI2 line for the function generator
        EXTI->IMR |= ((uint32_t)0x00000004); // unmasked
        EXTI->RTSR |= ((uint32_t)0x00000004); // Rising-edge trigger enabled

        // Set IRQ Priority for EXTI1 = IRQ6
        NVIC->IP[1] = ((uint32_t)0xFF00FFFF);

        // Enable IRQ Channel for EXTI1 = IRQ6
        NVIC->ISER[0] = (uint32_t)0X00000040;
    }
}

// get the direction of the signal to choose
uint16_t get_Signal(void)
{
    if ((EXTI->IMR & ((uint32_t)0x00000004)) != 0x00000000){
        return(choose_Signal_FG);
    }
    return(choose_Signal_555);
}

void myTIM2_Init()
{
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN; // Enable clock for TIM2 peripheral
    // Configure TIM2: buffer auto-reload, count up, stop on overflow,
    // enable update events, interrupt on overflow only
    TIM2->CR1 = ((uint16_t)0x008C);
    TIM2->PSC = myTIM2_PRESCALER; // Set clock prescaler value
    TIM2->ARR = myTIM2_PERIOD; // Set auto-reloaded delay
    TIM2->EGR = ((uint16_t)0x0001); // Update timer registers
    NVIC_SetPriority(TIM2_IRQn, 0); // Assign TIM2 interrupt priority = 0 in NVIC
    NVIC_EnableIRQ(TIM2_IRQn);// Enable TIM2 interrupts in NVIC
    TIM2->DIER |= TIM_DIER_UIE; // Enable update interrupt generation
}


void myTIM3_Init()
{
    RCC->APB1ENR |= RCC_APB1ENR_TIM3EN; // Enable clock for TIM2 peripheral
    // Configure TIM3: buffer auto-reload, count up, stop on overflow,
```

```
    // enable update events, interrupt on overflow only
    TIM3->CR1 = ((uint16_t)0x008C);
    TIM3->PSC = myTIM3_PRESCALER; // Set clock prescaler value: 48MHz/(47999+1) = 1 KHz
    TIM3->ARR = 100; // delay: 100 ms
    TIM3->EGR = ((uint16_t)0x0001); // Update timer registers
}


static void myADC_Init(void)
{
    // GPIOC Periph clock enable
    RCC->AHBENR |= RCC_AHBENR_GPIOCEN;  // RCC_AHBPeriph_GPIOC

    // ADC Periph clock enable
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;  // RCC_APB2Periph_ADC1

    // Configure ADC_PA5 as analog input
    GPIOA->MODER  &= 0xFFFFFF3FF; // clear the field
    GPIOA->MODER  |= 0x00000C00; // Set analog mode
    GPIOA->PUPDR &= 0xFFFFF3FF; // clear the field
    GPIOA->PUPDR |= 0x00000000; // Set no Pull-up/Pull-Down.

    // Set the ADC1 in continuous mode, Overrun mode, right data alignment with a resolution equal to 12
bits
    ADC1->CFGR1 = ADC_CFGR1_CONT | ADC_CFGR1_OVRMOD;

    // Select Channel 11 and set sampling rate
    ADC1->CHSELR |= ADC_CHSELR_CHSEL5;
    ADC1->SMPR &= ~((uint32_t)0x00000007);
    ADC1->SMPR |= ((uint32_t)0x00000007);

    ADC1->CR |= ((uint32_t)ADC_CR_ADEN); // Enable the ADC peripheral
    while(!(ADC1->ISR & ADC_CR_ADEN));  // Wait the ADRDY flag
}


static void myDAC_Init(void)
{
    // Enable GPIOA clock
    RCC->AHBENR |=  ((uint32_t)0x00020000); // RCC_AHBPeriph_GPIOA

    // Enable DAC clock
    RCC->APB1ENR |= ((uint32_t)0x20000000); // RCC_APB1Periph_DAC

    // Configure PA4 (DAC_OUT1) to analog mode
    GPIOA->MODER  &= 0xFFFFFCFF; // Clear the field
    GPIOA->MODER  |= 0x00000300; // Set analog mode
    GPIOA->PUPDR &= 0xFFFFFCFF; // Clear the field
    GPIOA->PUPDR |= 0x00000000; // Set analog mode
```

```
    // DAC Channel1 Init
    DAC->CR &= 0xFFFFFFF9; // clear TEN1 and BOFF1;
    DAC->CR |= 0x00000000; // set 0

    // Enable DAC Channel1
    DAC->CR |= 0x00000001; // then enable the DAC
}

//  Interrupt handler for 555 timer signal
void EXTI0_1_IRQHandler()
{
    double tempFreq = 0;
    if ((EXTI->PR & EXTI_PR_PR1) != 0) // Check if EXTI1 interrupt pending flag is set
    {
        TIM2->CR1 &= ~(TIM_CR1_CEN); // Stop the timer

        // If in first edge reset the count and start the timer
        if (timerTriggered == 0)
        {
            ElapsedTime = 0;
            timerTriggered = 1;

            TIM2->CNT = 0x00000000;         // Clear count register
            TIM2->CR1 |= TIM_CR1_CEN;       // Start the timer
        }

        // If in second edge stop the counter and calculate the frequency
        else
        {
            EXTI->IMR &= ~(EXTI_IMR_MR1);   // mask the EXTI1 interrupts
            ElapsedTime = TIM2->CNT;

            timerTriggered = 0;
            tempFreq = ((double)SystemCoreClock)/((double)ElapsedTime);
            Freq = (unsigned int)tempFreq;
            EXTI->IMR |= EXTI_IMR_MR1;      // unmask the EXTI1 interrupts

        }
        EXTI->PR = EXTI_PR_PR1;         // Clear the pending interrupt flag
    }

    // Check if EXTI0 interrupt pending flag is set
    if ((EXTI->PR & EXTI_PR_PR0) != 0)
    {
        if (get_Signal() == choose_Signal_555){
            choose_Signal(choose_Signal_FG); // alternate the interrupt flag to the other when its already set
        } else { // for the current one to the other when its already set for the current one
```

```
            choose_Signal(choose_Signal_555);
        }
        EXTI->PR = EXTI_PR_PR0;
    }

}

//  Interrupt handler for function generator signal
void EXTI2_3_IRQHandler()
{
    double tempFreq = 0;
    // Check if EXTI2 interrupt pending flag is set
    if ((EXTI->PR & EXTI_PR_PR2) != 0)
    {
        TIM2->CR1 &= ~(TIM_CR1_CEN); // Stop timer

        // If in first edge reset the count and start the timer
        if (timerTriggered == 0)
        {
            ElapsedTime = 0;
            timerTriggered = 1;

            TIM2->CNT = 0x00000000; // Clear the count register
            TIM2->CR1 |= TIM_CR1_CEN; // Start the timer
        }

        // If in second edge stop the counter and calculate the frequency
        else
        {
            EXTI->IMR &= ~(EXTI_IMR_MR2);   // mask the EXTI1 interrupts
            ElapsedTime = TIM2->CNT;

            timerTriggered = 0;
            tempFreq = ((double)SystemCoreClock)/ ((double)ElapsedTime);
            Freq = (unsigned int)tempFreq;

            EXTI->IMR |= EXTI_IMR_MR2;      // unmask the EXTI1 interrupts
        }
        EXTI->PR = EXTI_PR_PR2; // Clear the pending interrupt flag
    }

}


void TIM2_IRQHandler()
{
    // Check if the update interrupt flag is set
    if ((TIM2->SR & TIM_SR_UIF) != 0)
    {
```

```
        trace_printf("\nOverflow!\n");

        TIM2->SR &= ~(TIM_SR_UIF);  // Clear the update interrupt flag
        TIM2->CR1 |= TIM_CR1_CEN;   // Restart the stopped timer
    }
}

// OLED Display Functions
void oled_refresh(void)
{
    // Buffer size = at most 16 characters per PAGE + terminating '\0'
    unsigned char Buffer[17];

    // FORMAT: snprintf(char *str, size_t size, const char *format, ...);
    snprintf( Buffer, sizeof( Buffer ), "Res: %5u Ohms", Res );
    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */
    oled_write_text(0, 0, Buffer);
    // print the LED display from buffer

    snprintf( Buffer, sizeof( Buffer ), "Freq: %5u Hz", Freq );
    /* Buffer now contains your character ASCII codes for LED Display
       - select PAGE (LED Display line) and set starting SEG (column)
       - for each c = ASCII code = Buffer[0], Buffer[1], ...,
           send 8 bytes in Characters[c][0-7] to LED Display
    */
    oled_write_text(0, 1, Buffer);
    // print the LED display from buffer

    // print the Name and Current signal
    if (choose_Signal_FG == get_Signal()){
            oled_write_text(0, 3, "Function" );
            oled_write_text(0, 4, "Generator" );
        } else {
            oled_write_text(0, 3, "555     ");
            oled_write_text(0, 4, "Timer    ");
        }

    oled_write_text(0, 6, "Emil Vu");
    oled_write_text(0, 7, "Minh Nguyen");

    /* Wait for ~100 ms (for example) to get ~10 frames/sec refresh rate
       - You should use TIM3 to implement this delay (e.g., via polling)
    */
    oled_show();
```

```
    TIM3->CNT = 0; // clear the timer
    TIM3->ARR = 100; // 100ms delay
    TIM3->EGR = ((uint16_t)0x0001); // update the register
    TIM3->CR1 |= TIM_CR1_CEN; // start thetimer
    while ((TIM3->SR & TIM_SR_UIF) == 0) {;} // Wait for the delay
    TIM3->SR &= ~(TIM_SR_UIF); // Clear the update interrupt flag
}



void oled_Write_Cmd( unsigned char cmd )
{
    GPIOB->ODR |= (1 << 6);// make PB6 = CS# = 1
    GPIOB->ODR &= ~(1 << 7); // make PB7 = D/C# = 0
    GPIOB->ODR &= ~(1 << 6); // make PB6 = CS# = 0
    oled_Write( cmd );
    GPIOB->ODR |= (1 << 6); // make PB6 = CS# = 1
}

void oled_Write_Data( unsigned char data )
{
    GPIOB->ODR |= (1 << 6); // make PB6 = CS# = 1
    GPIOB->ODR |= (1 << 7); // make PB7 = D/C# = 1
    GPIOB->ODR &= ~(1 << 6); // make PB6 = CS# = 0
    oled_Write( data );
    GPIOB->ODR |= (1 << 6); // make PB6 = CS# = 1
}


void oled_Write( unsigned char Value )
{

    /* Wait until SPI1 is ready for writing (TXE = 1 in SPI1_SR) */

    while((SPI1->SR & SPI_SR_TXE) != SPI_SR_TXE){
        // Waiting for the flag
        ;
    }

    /* Send one 8-bit character:
      - This function also sets BIDIOE = 1 in SPI1_CR1
    */
    HAL_SPI_Transmit( &SPI_Handle, &Value, 1, HAL_MAX_DELAY );

    /* Wait until transmission is complete (TXE = 1 in SPI1_SR) */

    while((SPI1->SR & SPI_SR_TXE) != SPI_SR_TXE){
        // Waiting until TXE = 1
        ;
```

```
    }
}

// writes a single character to a specific position in the OLED buffer.
void oled_write_character(unsigned int x, unsigned int y, unsigned int c)
{
    static unsigned int column, offset;

    // Calculate the offset in the buffer for the character's position.
    // y << 7 shifts the row (y) into the correct page offset.
    // x << 3 shifts the column (x) for an 8-column wide character (1 byte per column).

    offset = (y << 7) + (x << 3);

    // Write the character's column data (8 bytes) to the OLED buffer.
    for (column = 0; column <= 7; column++){
        /*
        x: Horizontal position (column) for the character.
        y: Vertical position (row/page) for the character.
        c: ASCII code of the character to display.
        */
        oled_buffer[offset + column] = Characters[c][column];
    }
}

// writes a string of text starting at a specific position on the OLED.
void oled_write_text(unsigned int x, unsigned int y, char *s)
{
    while (*s != 0){ // Loop until the end of the string ('\0'), "s" is the pointer to the pos of string
        // Write the current character to the OLED buffer.
        oled_write_character(x, y, *s);
        s++; // Move to the next character in the string.
        x = x + 1; // Move to the next character position (increment column).

        if ( x > 16 ){ // Prevent writing beyond 16 characters in a row.
            break;
        }
    }
}

// Sends the entire buffer to the OLED hardware to update the display.
void oled_show(void)
{
    unsigned int p = 0; // Pointer to track the buffer position
    unsigned int n;

    for (unsigned int i = 0; i < 8; i++){ // // Iterate through all 8 rows of the OLED display.
        oled_Write_Cmd( 0xB0 | ( i & 0x0f ));
```

```
        // Set the column start address to 0 (low and high nibble).
        oled_Write_Cmd( 0x02 );
        oled_Write_Cmd( 0x10 );

        // Write 128 bytes (1 row of the display) from the buffer
        for ( n = 0; n < 128; n ++ ){
            oled_Write_Data( oled_buffer[p]); // Send one byte of data.
            p++; // Move to the next byte in the buffer.
        }
    }
}

void oled_config(void)
{
    // Enable clock for GPIOB and SPI1
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;

    // Configure PB3 (SCLK) and PB5 (MOSI) for SPI in alternate function mode
    GPIOB->MODER &= ~((3 << (3 * 2)) | (3 << (5 * 2))); // Clear mode bits
    GPIOB->MODER |= (2 << (3 * 2)) | (2 << (5 * 2));    // Set to alternate function
    GPIOB->AFR[0] |= (0x0 << (3 * 4)) | (0x0 << (5 * 4)); // Alternate function AF0

    // Configure PB4 (Reset), PB6 (CS), PB7 (DC) as output
    GPIOB->MODER &= ~((3 << (4 * 2)) | (3 << (6 * 2)) | (3 << (7 * 2))); // Clear mode bits
    GPIOB->MODER |= (1 << (4 * 2)) | (1 << (6 * 2)) | (1 << (7 * 2));    // Set to output
    GPIOB->OTYPER &= ~((1 << 4) | (1 << 6) | (1 << 7)); // Push-pull
    GPIOB->OSPEEDR |= (3 << (4 * 2)) | (3 << (6 * 2)) | (3 << (7 * 2)); // High-speed
    GPIOB->PUPDR &= ~((3 << (4 * 2)) | (3 << (6 * 2)) | (3 << (7 * 2))); // No pull-up/pull-down

// Don't forget to enable GPIOB clock in RCC
// Don't forget to configure PB3/PB5 as AF0
// Don't forget to enable SPI1 clock in RCC

    SPI_Handle.Instance = SPI1;

    SPI_Handle.Init.Direction = SPI_DIRECTION_1LINE;
    SPI_Handle.Init.Mode = SPI_MODE_MASTER;
    SPI_Handle.Init.DataSize = SPI_DATASIZE_8BIT;
    SPI_Handle.Init.CLKPolarity = SPI_POLARITY_LOW;
    SPI_Handle.Init.CLKPhase = SPI_PHASE_1EDGE;
    SPI_Handle.Init.NSS = SPI_NSS_SOFT;
    SPI_Handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    SPI_Handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    SPI_Handle.Init.CRCPolynomial = 7;

//
```

```
// Initialize the SPI interface
//
    HAL_SPI_Init( &SPI_Handle );

//
// Enable the SPI
//
    __HAL_SPI_ENABLE( &SPI_Handle );


    /* Reset LED Display (RES# = PB4):
       - make pin PB4 = 0, wait for a few ms
       - make pin PB4 = 1, wait for a few ms
    */
    GPIOB->ODR &= ~GPIO_PIN_4; //Reset Oled screen 0
    for (int i = 0; i < 65500; i++); // wait for a few ms
    GPIOB->ODR |= GPIO_PIN_4; // Reset Oled screen -> 1
    for (int i = 0; i < 65500; i++); // wait for a few ms


//
// Send initialization commands to LED Display
//
    for ( unsigned int i = 0; i < sizeof( oled_init_cmds ); i++ )
    {
        oled_Write_Cmd( oled_init_cmds[i] );
    }

    /* Fill LED Display data memory (GDDRAM) with zeros:
       - for each PAGE = 0, 1, ..., 7
           set starting SEG = 0
           call oled_Write_Data( 0x00 ) 128 times
    */
     oled_show();
}



#pragma GCC diagnostic pop

// ----------------------------------------------------------------------
```

References

[1] B. Sirna, K. Kelany, and D. Rakhmatov, ECE 355: Microprocessor-Based Systems Laboratory Manual, University of Victoria, 2023.

[2] STMicroelectronics, STM32F0xxx Cortex-M0 Programming Manual, PM0215, Rev. 1, Apr. 2012. [Online]. Available: https://www.st.com

[3] STMicroelectronics, STM32F0x1/STM32F0x2/STM32F0x8 Advanced ARM-Based 32-Bit MCUs Reference Manual, RM0091, Rev. 5, Jan. 2014. [Online]. Available: https://www.st.com